HW2 Problem Set

ECS 240

Due: TBD

This problem set contains 5 problems. As with HW1, problems 1, 3, and 5 are coding problems, and problems 2 and 4 are pure theory problems. There is a template in `src/` which you can modify for the programming problems. Solutions to the theory problems should be written in LaTeX.

---

1. In class (Lecture 2, Part 1) we discussed that for compilers, there are three different languages involved – each with their own semantics, and a valid compiler must be consistent with all three of them. Namely:

   - The source language S (language in which we write the program)
   - The meta language M (language in which we write the compiler)
   - The target language T (language to which we compile the program).

   This problem asks you to complete the definition of a *correct* compiler (one which implements the semantics), based on these three semantics.

   (a) Complete the definition below based on the template that we started in class. **Hint:** you may find it helpful to reference the definition of when an interpreter satisfies a semantics that we saw in class (Lecture 1). Your definition should follow a similar idea.

   Defiition 1. *Let M be the semantics (set of $(\texttt{INPUT}, \texttt{prog}, \texttt{OUTPUT})$) for the metalanguage. Let S be the semantics of the source language. Let T be the semantics of the target language.*

   A compiler C is a program in the metalanguage M.

   A compiler C ***is correct****, or* ***satisfies*** *the semantics, if . . . TODO: Complete the definition here.*

   (b) Use your definition to prove the following: If C_1 is a correct compiler from S to T, and C_2 is a correct compiler from T to U, then the compiler C defined by the following pseudocode is a correct compiler from S to U:

```
Compiler C from S to U:
On input prog:
- Run C1: set out1 = C1(prog).
- Run C2 on the result: set out2 = C2(output1).
- Return out2.
```

You may assume that the meta-language M runs the pseudocode above in the way that you would expect (i.e., it runs C_1, followed by C_2, and returns the result, similar to how `Seq` works for `RandProg`.)

Note: This part is essentially showing that compiling a program through an intermediate representation works out. In this case, the intermediate representation is T and the overall compiler is from S to U.

(c) Use your definition to prove the following: If C is a correct compiler from S to T, and I_T is an interpreter for T that implements the semantics, then running C, followed by running I_T on the result, results in a correct interpreter for S. That is:

```
Interpreter I_S for S:
On input (INPUT, prog):
- Run C: set out = C(prog).
- Run I_T((INPUT, out)).
```

You may assume that the meta-language M runs the pseudocode above in the way that you would expect (i.e., it runs C followed by I_T).

(d) Define the "do nothing" compiler from S to S to be a compiler which simply takes the input program, and returns it without any changes. This is a program in the metalanguage M such that

$$(\texttt{INPUT}, \texttt{do_nothing}, \texttt{OUTPUT})$$

is valid if and only if $\texttt{INPUT} = \texttt{OUTPUT}$.

Use your definition to prove or disprove the following: Regardless of the semantics of S, and even if S is nondeterministic, a do-nothing compiler is a valid compiler from S to itself.

2. This problem will ask you to expand the basic `RandProg` compiler from class to add some syntactic sugar. First, copy in any necessary code involving `RandProg` from the Lecture 2, Part 1 source code. Then complete the following (this was an exercise that we skipped in class):

(a) First, fix our compiler from class to produce runnable code. To do this, you will need to have it generate a `main` function, and you may also need to inline (or include with the compiled target code) the utility functions we used from `util.rs`. The compiler should now produce a single `String` for a `main.rs` source file, which can be run directly.

Write a unit test to see that it is working.

(b) Add syntactic sugar to our language for 4-bit integers, which support equality, less-than, and addition. Adapt and structure the compiler in such a way that the 4-bit integer logic is compiled out, and the 4-bit integers are reduced to Booleans.

(c) Add at least two unit tests to ensure that your new compiler is working.

Function and type names: For part (a), use the same `RandProg` and compiler from class. For part (b), please define a Rust enum `RandProgP2` for the syntax with the syntactic sugar, `struct RandProgP2Compiler` for the compiler, which supports methods

```
impl RandProgP2Compiler {
    pub fn new() -> Self
    pub fn compile(&mut self, prog: &RandProgP2) -> String
}
```

For the unit tests in (a) and (c), please name these `test_2a`, `test_2c_1`, and `test_2c_2`. (You may add other unit tests with other names.)

3. This problem will ask you to expand the basic `RandProg` to add something resembling platform-specific code. Use the same `RandProg` enum from Problem 2. Then, complete the following (this was an exercise that we skipped in class):

   (a) Add some "platform-specific code" for random number generation by allowing our language to be configured to different pseudorandom number generators at the back-end: (1) Rust thread RNG (default Rust random generator) (2) Always return Heads (3) Always return Tails (4) On Unix systems, use Unix's `/dev/random` for RNG rather than the built in Rust thread RNG.

   (b) Add at least three unit tests to ensure that your new compiler is working.

   Function and type names: Use `RandProg` for the syntax, and `RandProgP3Compiler` for the compiler. To get this to work, you will need to add a `RandProgRngConfig` enum to your compiler that tells which platform to compile for:

   ```
   #[derive(Debug, Copy, Clone)]
   pub enum RandProgRngConfig {
       DefaultRng,
       AlwaysHeads,
       AlwaysTails,
       DevRandom,
   }
   ```

   Then, your compiler should have two different constructors:

   ```
   impl RandProgP3Compiler {
       pub fn new() -> Self // Use default Rng
       pub fn new_with_rng(RandProgRngConfig) -> Self
       pub fn compile(&mut self, prog: &RandProg) -> String
   }
   ```

   Please name your unit tests `test_3b_1`, `test_3b_2`, and `test_3b_3`. (You may add other unit tests with other names.)

4.

5.

Submission instructions:

- Upload your solutions (as a PDF) and your code (as a `.zip` file, as with HW0: including the entire repository directory including config files like `Cargo.toml`, but excluding `target/`) in Gradescope.
- Your PDF should be a single file `hw2.pdf`. If you use this LaTeX template to create your solutions, please remove the problem statements and include only your solutions.
- The `main.rs` should contain your solutions to all coding parts (including functions and types as described above). Please do not modify function signatures. You are welcome to add additional functions and tests – if you do so, please include all additional tests only at the bottom of the file.
- You may add utility modules, like `mod util;` then `use util::...`, if you feel that you need them; but you should put all the main functions with the names as above in `main.rs`. Please double check on Piazza before importing any external crates in your `Cargo.toml`.
- Make sure that `cargo build`, `cargo clippy`, `cargo test`, and `cargo fmt` (or `cargo fmt --check`) pass without any errors.